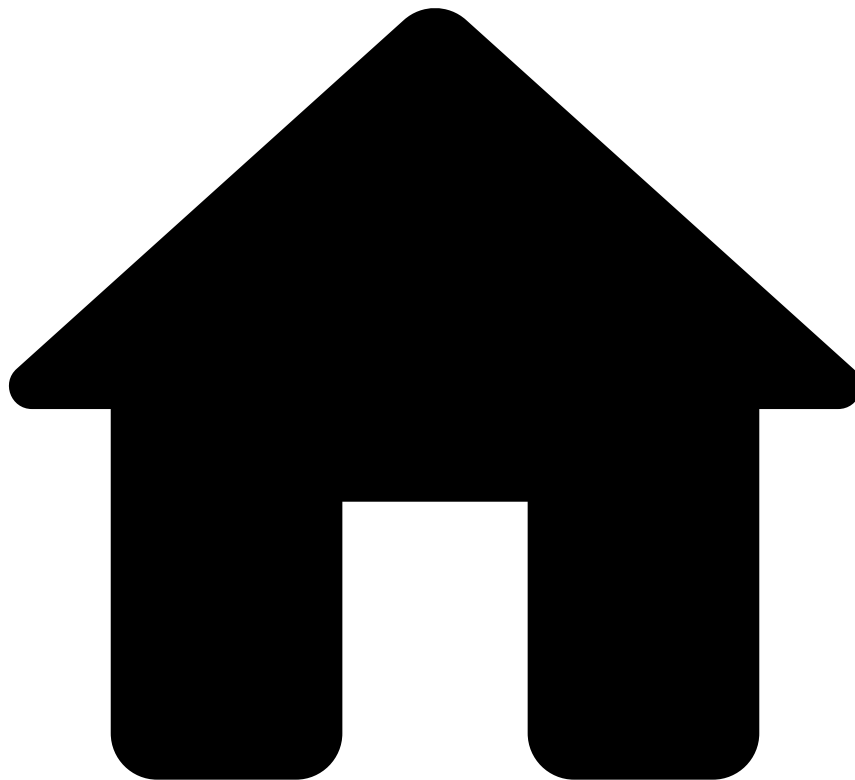


•



- [Creating a Data Science project](#)
- Using the Data Science Framework (DSF)

On this page

Using the Data Science Framework (DSF)

Was this page helpful? 

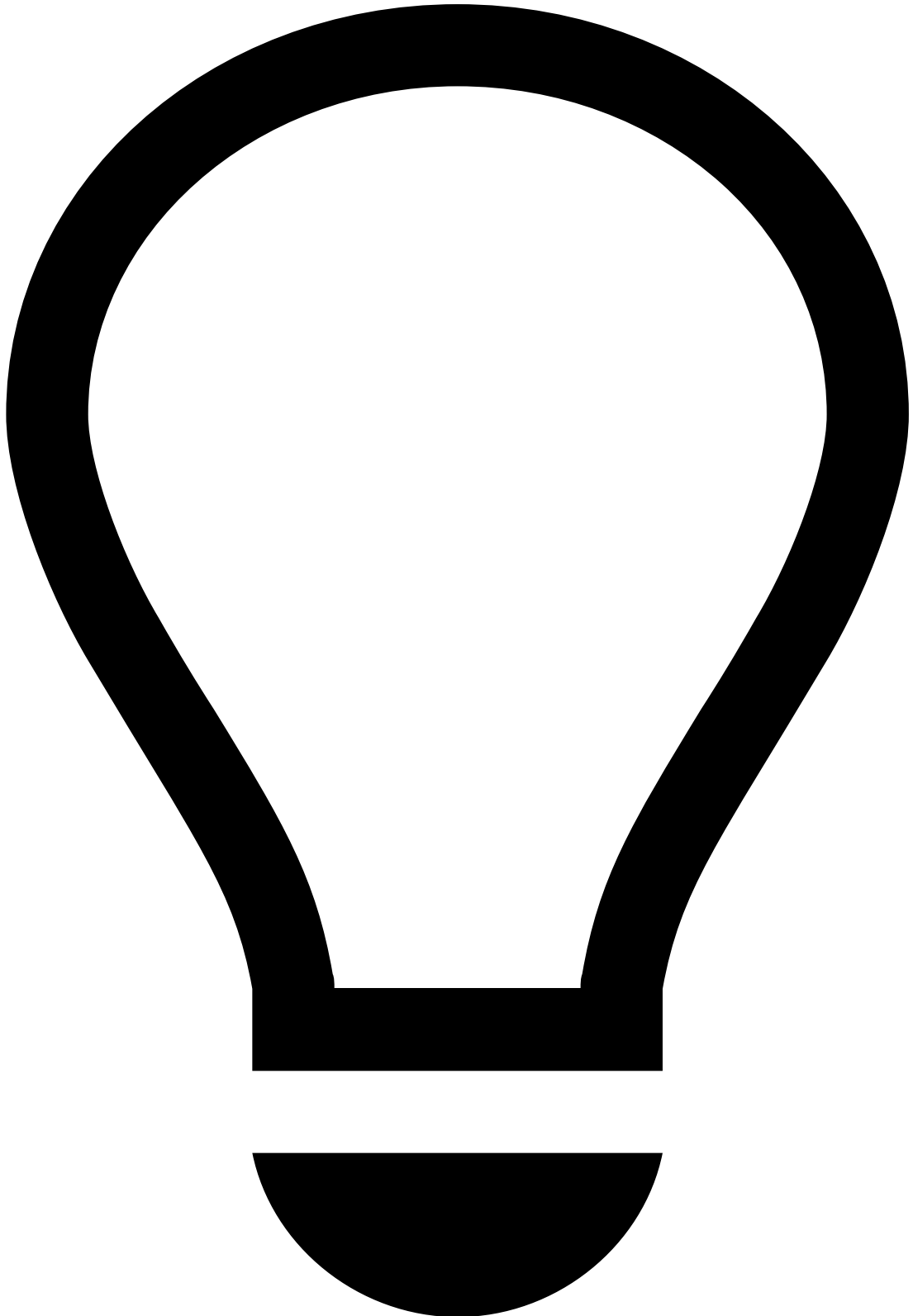
The Data Science Framework (DSF) is a dedicated environment used in risk assessment strategies for building, testing, and evaluating rules and models according to the Data Science Loop.

It allows for initiating Data Science (DS) jobs, either via Pulse or JupyterLab, that are then processed through the Pulse application. Before diving into the importance and impact of tuning jobs, learn more about the DSF and DS jobs [here](#).

Why tune DS jobs?

Tuning DS jobs is the process of optimizing Spark configurations used by the cluster to handle specific workloads efficiently. This helps prevent:

- Executions from hanging due to the lack of requested resources.
- Executions crashing due to cheap job configurations.
- Resource starvation caused by greedy job configurations.



tip

Choosing the appropriate configuration is **trade-off between what's available and what's needed**.

Note that there isn't a "one-size-fits-all" configuration. Heavier jobs will take advantage of more generous configurations, while lighter jobs will perform well with more sensible ones.

Additionally, clusters are shared by all team members, so guaranteeing that everyone is mindful of their job configurations is key to avoid concurrency issues and leverage the resources appropriately.

Jobs can be tuned either within Pulse (using Job Profiles) or in JupyterLab (by providing the Spark configuration when opening a session).

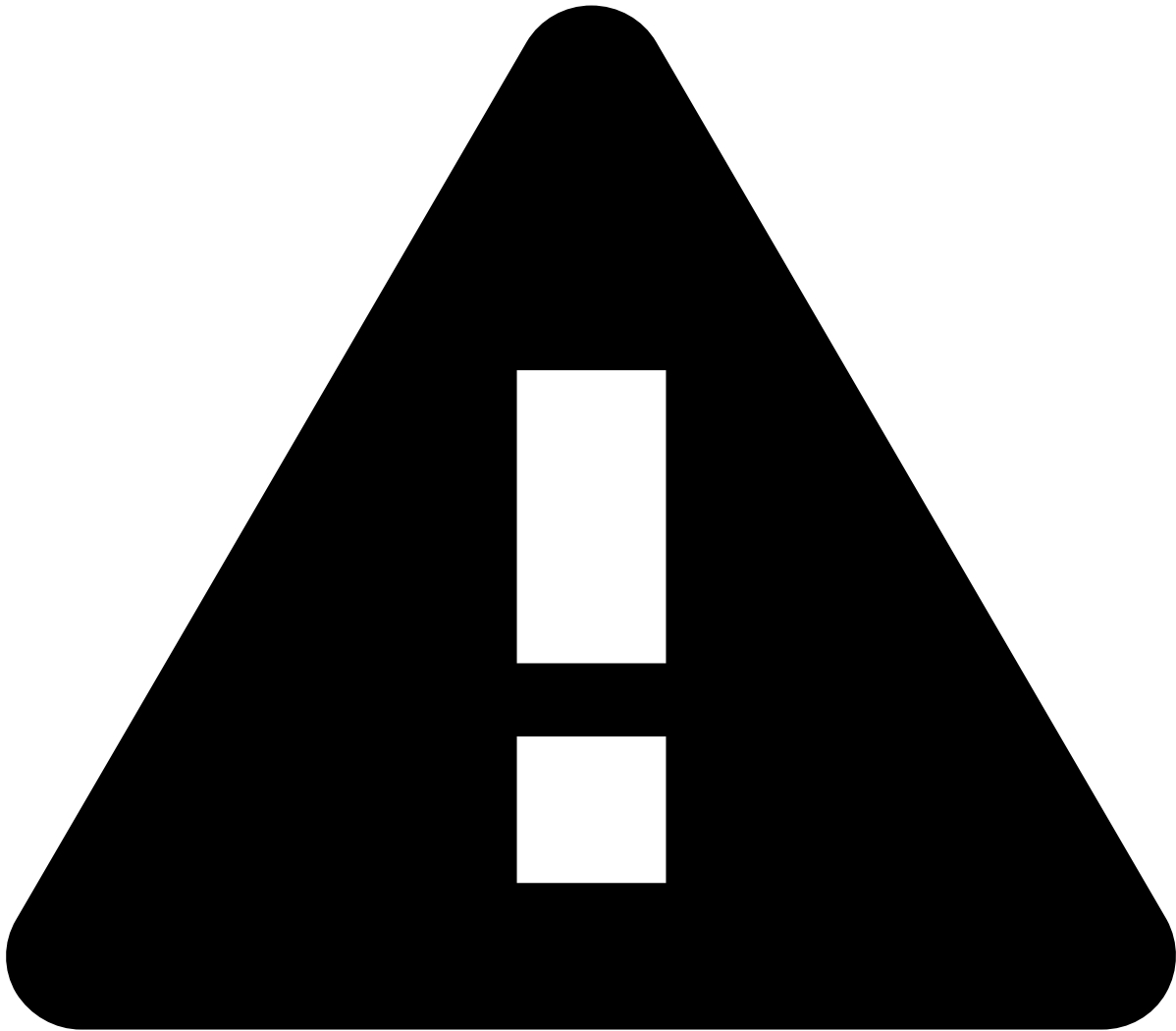
Impact of tuning in the DS job life cycle

To better understand how job configurations affect the life cycle of a DS job, it is important to consider the [scaling capacity](#) and how [resources are allocated and shared](#).

Scaling capacity

Scaling refers to the cluster's capacity to add and remove task nodes from the pool of resources at any time depending on workload. It occurs automatically based on demand given by the number of executors that are waiting to be allocated.

The scale-up strategy follows a tiered approach: the higher the demand (i.e. jobs pending/waiting for allocation), the more capacity is allocated. These tiers can be classified based on how fast more capacity is added (e.g., light, moderate, high, very high).



Capacity limits

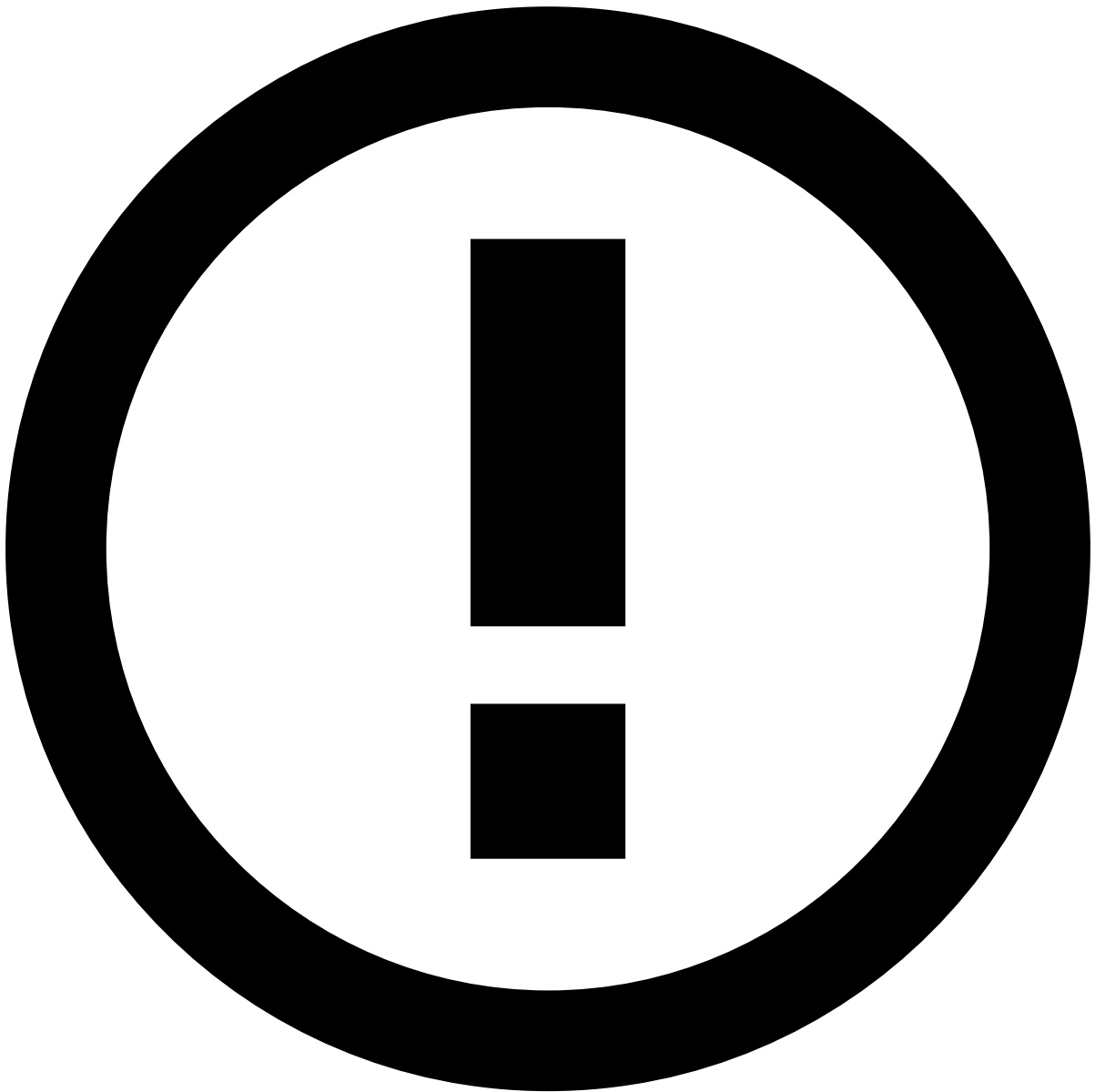
There is an upper bound to the maximum capacity attributed to you based on your contract with Feedzai.

Resource allocation and sharing^â

Choosing adequate capacity for a job is important to allow for optimal execution times and sharing that capacity with other people in the team. To better understand how resources are allocated and shared within the cluster, it's important to know [how jobs allocate capacity](#) and [what it means to have running Spark sessions](#).

How do jobs allocate capacity?^â

When you submit a Pulse job or open a Spark session, the EMR (Elastic MapReduce) cluster registers the request and allocates resources dynamically depending on the workload. This means that if you request a job with a minimum of 1 and a maximum of 8 executors, the job begins executing as soon as the cluster is able to allocate 1 executor. If during the job execution more cluster resources become available (e.g., because other jobs have finished or the cluster has scaled up), the cluster allocates additional executors, up to a maximum of 8.



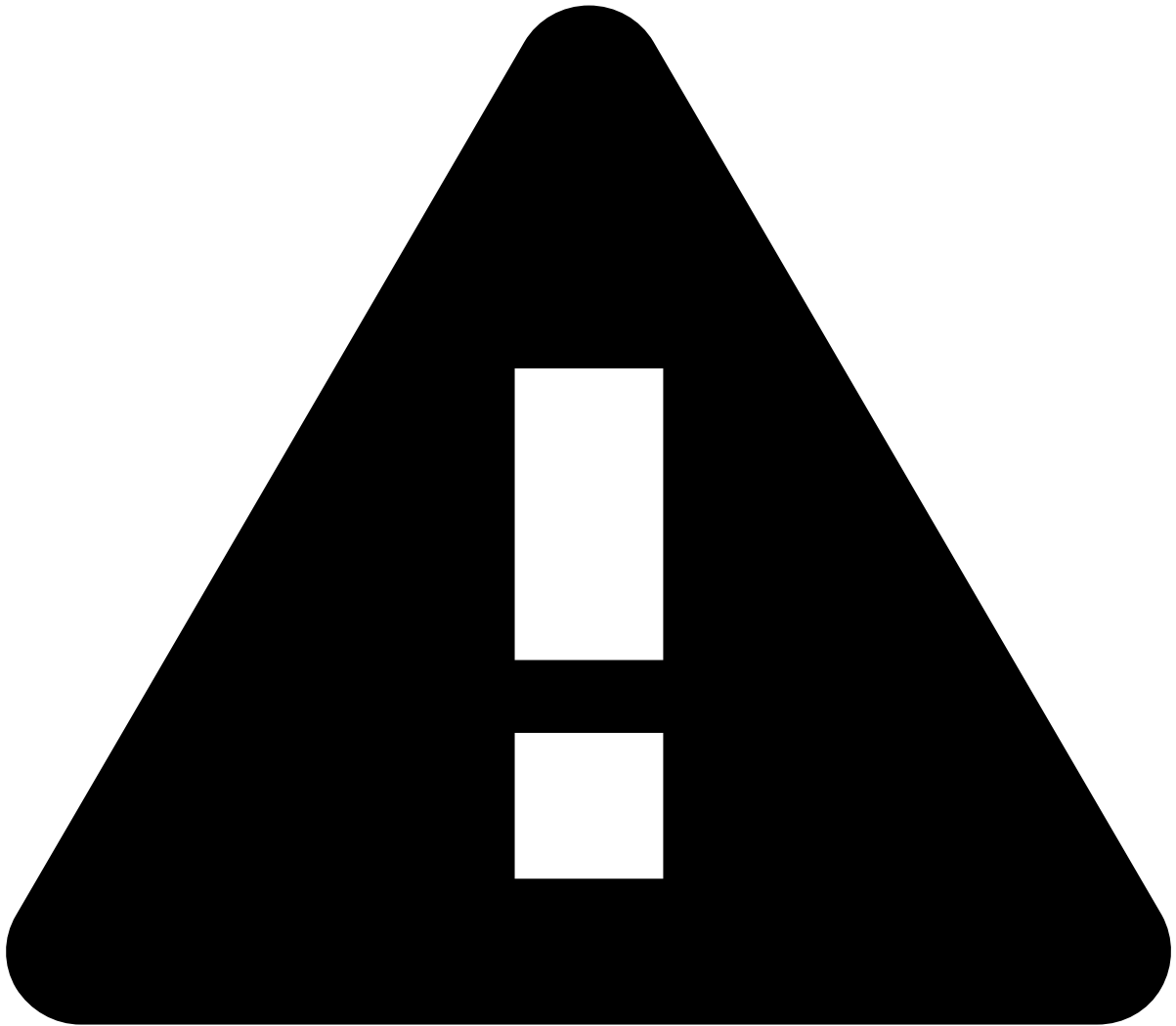
info

Note that executors are main processing units for jobs and are indivisible. This means that while executors for the same job can potentially “live” in a different cluster node, each executor alone must fit entirely in a given cluster node.

Refer to the [What are executors?](#) section for more details on these processing units.

On the other hand, if dynamic allocation is disabled, the job only begins executing when the cluster is able to allocate all the executors requested in the job configurations. For example, if you request a job with 8 executors (each with 12GB and 4 cores), the job will start once the cluster is able to fit those 8 executors within the pool of cluster resources available at that moment.

Refer to [Applying configurations in JupyterLab](#) and [Applying configurations in Pulse](#) for details on how to configure jobs/sessions, respectively.

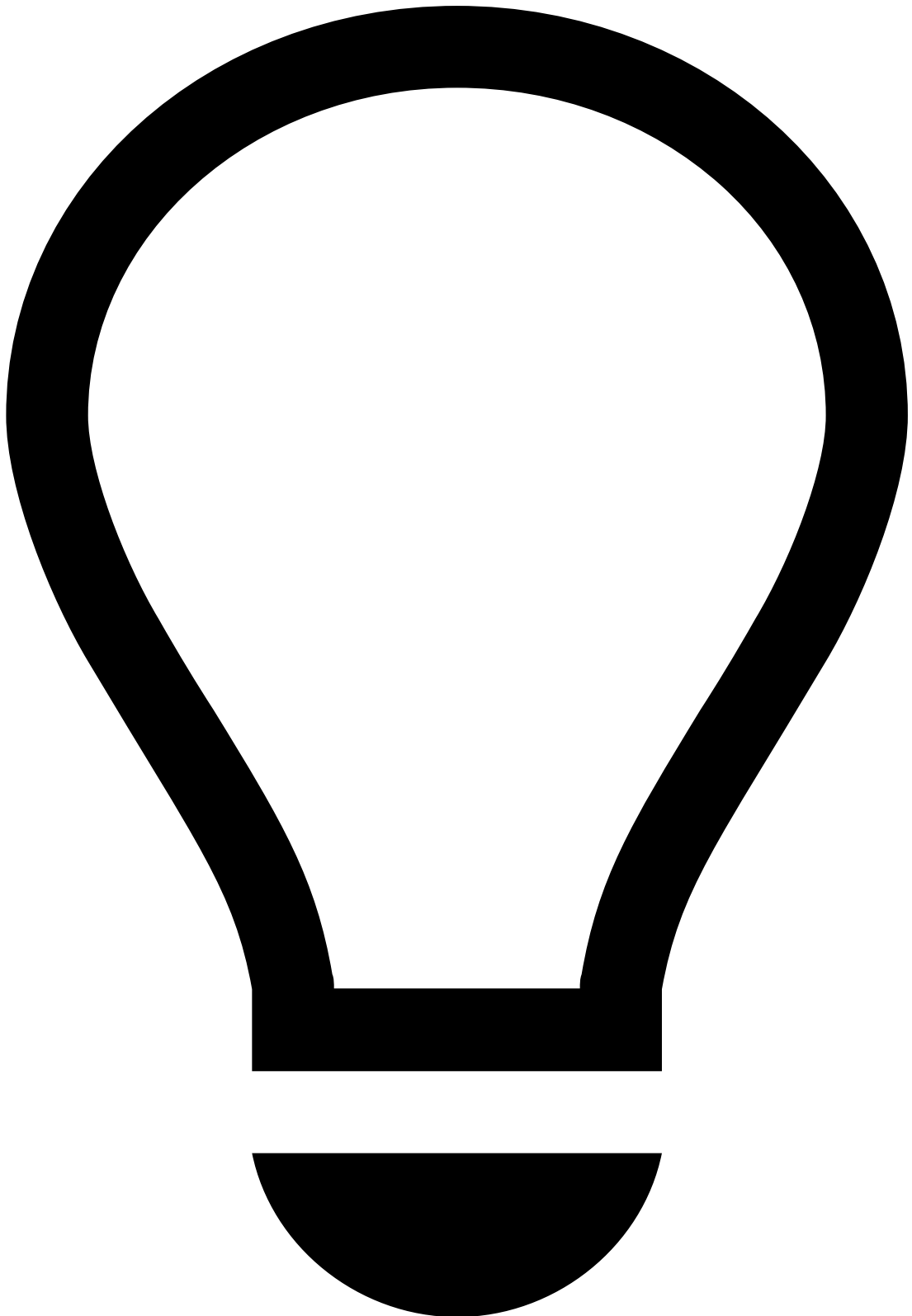


warning

Disabling dynamic allocation is only recommended when debugging issues with the cluster.

Depending on the number of executors being requested by all the jobs registered in the cluster at a given moment (including jobs waiting for resources to begin execution, and dynamic allocation jobs already executing and requesting more executors), the cluster can scale up and add more nodes to its pool of resources.

In contrast, if the cluster resources are idle for a certain amount of time (e.g., because there are few jobs running) it can scale down and remove cluster nodes from the pool of resources.



What does this mean?

Both the cluster resource usage (i.e. all jobs from all users using the cluster at the same time) and the cluster total pool of resources (i.e. if the cluster is scaled up or scaled down) have an impact on the duration of your job. Therefore, the exact same job might take a different amount of time to complete on separate occasions.

When you submit your job, you can check its status (i.e. if it's running or waiting for resources) in the **YARN Web UI**. There, you can also find the total and available pool of cluster resources. These metrics can help you understand why your job hasn't started yet (e.g., if there isn't enough memory or cores to satisfy your job configurations).

You can also use the **YARN Web UI** to check the resource usage of your running jobs to make sure they aren't exhausting most of the cluster resources and adhere to fair cluster usage.

What does it mean to have running Spark sessions? 📖

Spark sessions are created in JupyterLab and represent the connection between your notebook and the cluster. Refer to the [Interacting with Pulse via JupyterLab](#) page for additional information, including a list of available notebooks.

Note that once created, Spark sessions remain active until you either:

- Close them using `spark_session.stop()` .
- OR
- Kill the notebook kernel.



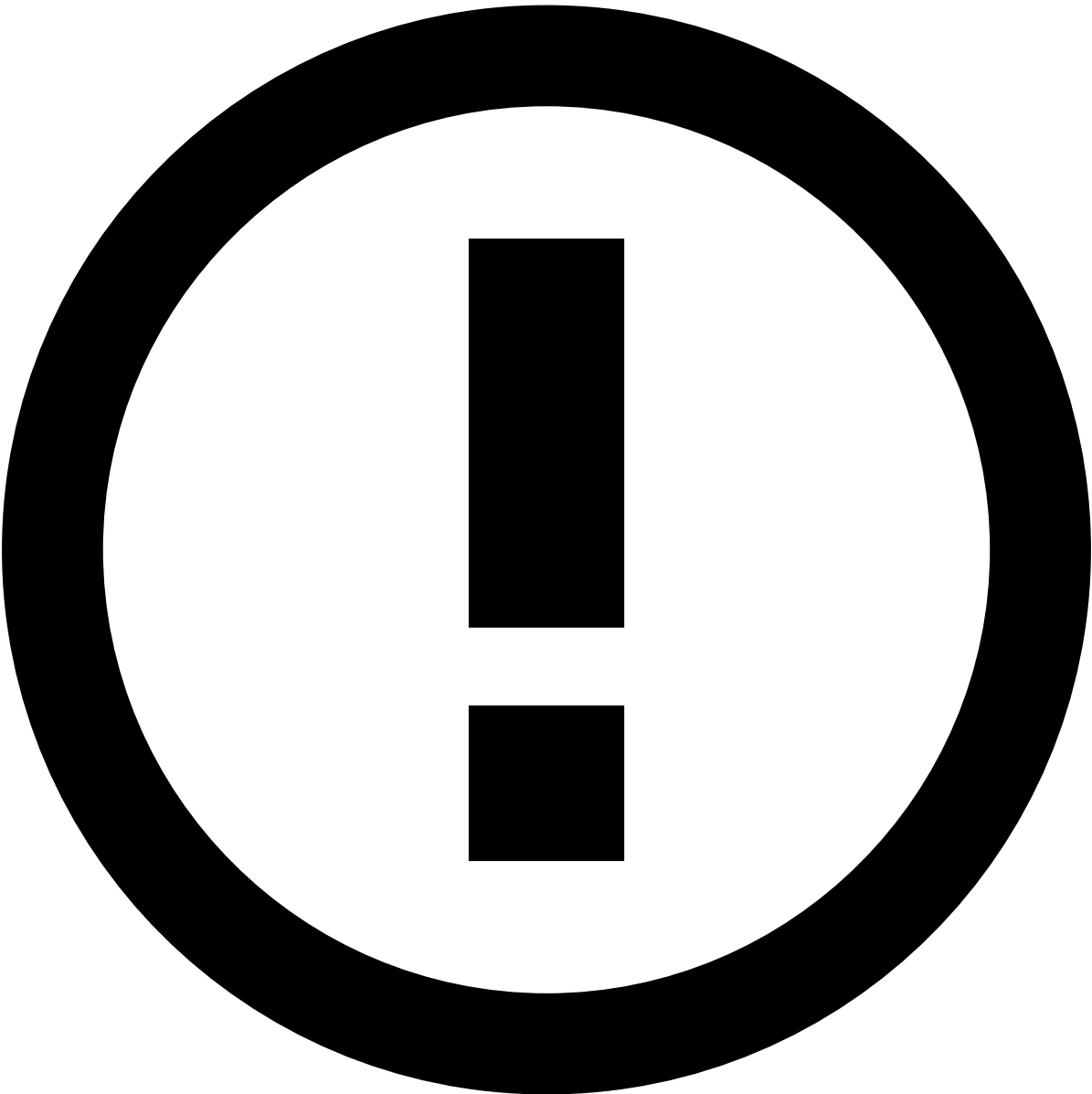
warning

Be mindful of your active Spark sessions, and make sure you manually stop them if they're no longer needed. Hanging (idle) sessions can occupy cluster resources and impact the other cluster users' experience.

In some cases, Spark sessions that are idle for a long period are automatically closed, but this depends on the cluster's configuration.

Common scenarios

Here are some scenarios where tuning Spark configurations is most commonly needed and what changes they require:



info

The following table provides guidance in terms of how different parameters could be configured, but choosing the appropriate configuration is **trade-off between what's available and what's needed**. Be mindful of job configurations to avoid concurrency issues and leverage the resources appropriately.

There isn't a "one-size-fits-all" configuration. Heavier jobs will take advantage of more generous configurations, while lighter jobs can be executed in an acceptable timeframe with less demanding configurations, and without compromising other user's jobs.

Scenario	Considerations
Dataset analysis and rule evaluations	This scenario doesn't require you to limit the number of executors.

Scenario	Considerations
Model training	This is a <u>single-task/executor</u> job, so: • The number of cores per executor equals the number of cores per task. • The rule-of-thumb for job memory and cores per executor is 1/4 of the task node size for both. This allows for running four models in parallel in just one task node, which complies with the default HPT notebooks (that launch four model training jobs simultaneously).  note Even though model training jobs are single-task/executor jobs, you must allocate more than one CPU per task using <code>spark.task.cpus</code> . This is set by default to <code>1</code> . However, if you don't override this default, you're essentially running a job with X executors but only <code>1</code> will be used (which is a waste of resources). Since model training is a single-task/executor job, make sure the task gets all the executor cores by matching the <code>spark.task.cpus</code> to the <code>spark.executors.cores</code> . Essentially, this allows you to increase the number of cores per executor in order to leverage model training multi-threading (as long as that property remains aligned with the number of CPUs per task).
Plan executions	In this scenario: • Memory-heavy jobs benefit from parallelization. • Both the cores and memory per executor should be maximized.
Data exploration/manipulation	In this scenario: • The necessary configurations vary depending on the data and type of exploration/manipulation that will be performed.

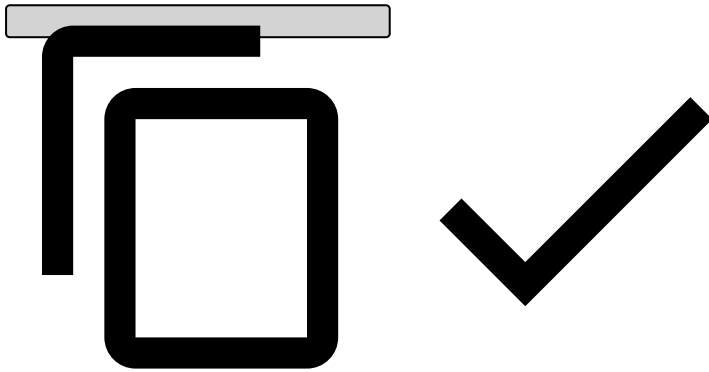
Scenario	Considerations
	• A mixture of approaches is recommended.

Applying configurations in JupyterLab

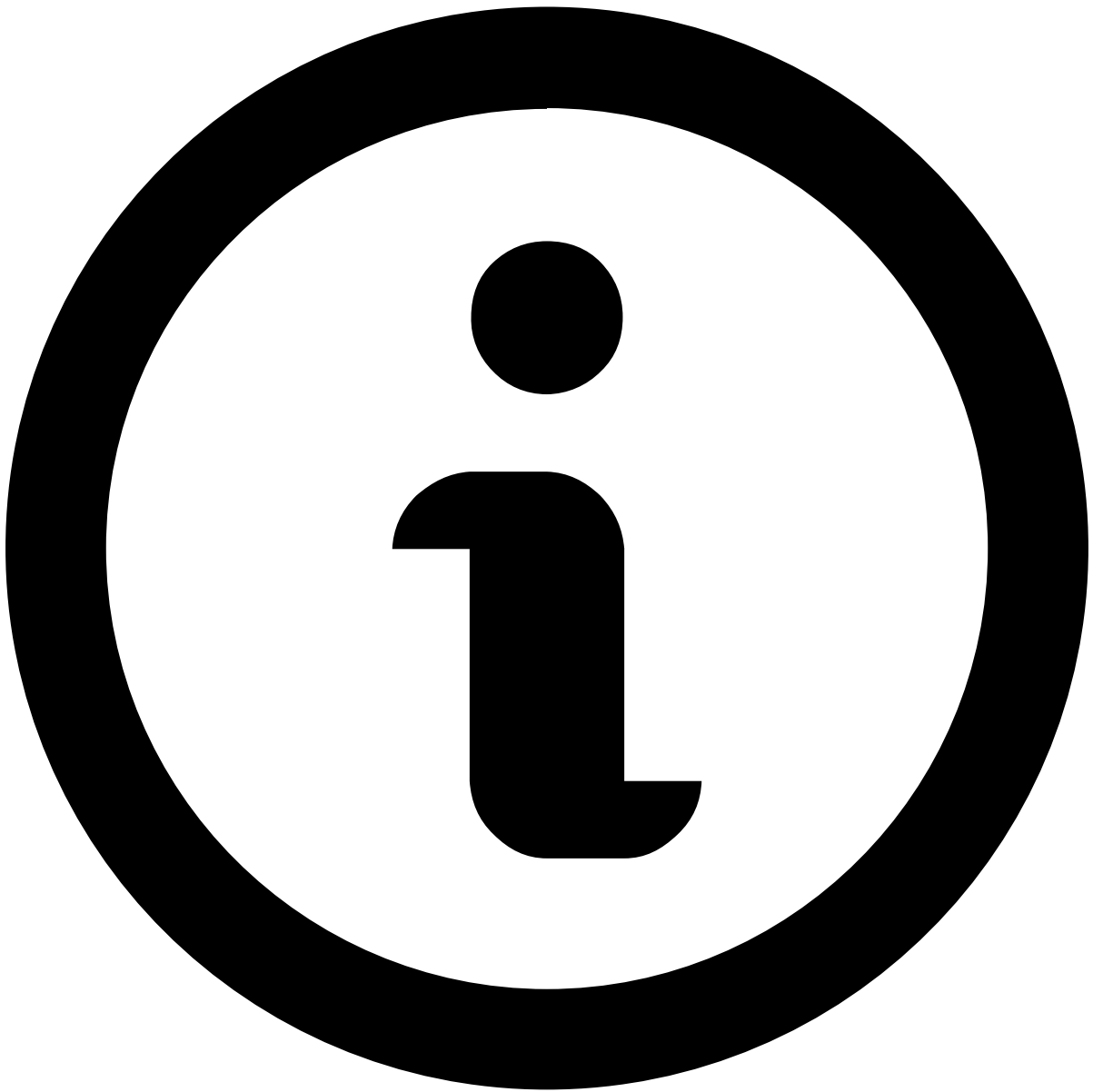
When creating Spark sessions via JupyterLab (e.g., when using notebooks), you need to provide the Spark configuration at the beginning:

```
# Spark Session Parameters
SPARK_PARAMETERS = {
    "spark.executor.cores": "7",
    "spark.executor.memory": "8g",
    "spark.executor.memoryOverhead": "5g",
    "spark.dynamicAllocation.enabled": True,
    "spark.dynamicAllocation.initialExecutors": "2",
    "spark.dynamicAllocation.minExecutors": "1",
    "spark.dynamicAllocation.maxExecutors": "6",
    "spark.dynamicAllocation.executorIdleTimeout": "300s",
}

# Get SparkSession with default Pulse's configuration
spark = feedzai.get_spark_session(SPARK_PARAMETERS)
```



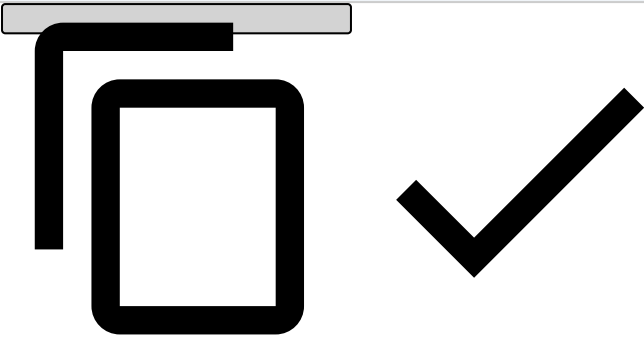
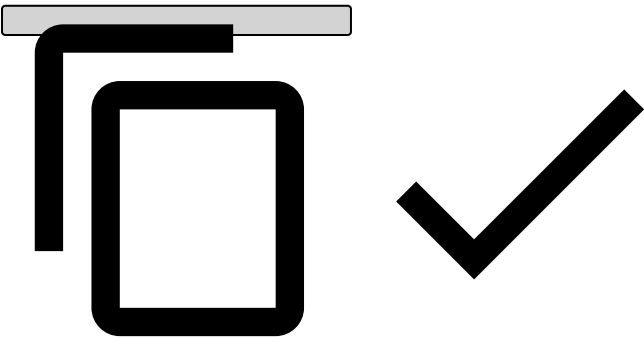
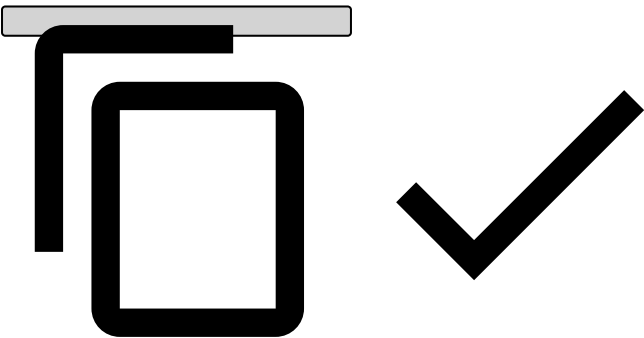
The configuration examples below can be used to configure your Spark jobs:



note

Ensure you replace the values for each as needed.

Configuration aspect	Example
Cores per executor	<pre>SPARK_PARAMETERS = { "spark.executor.cores": "7", }</pre>

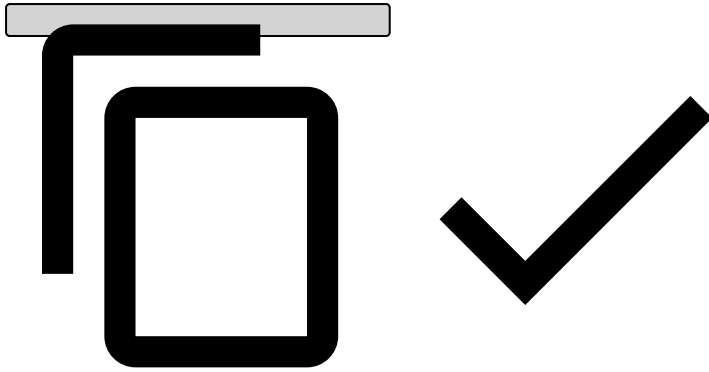
Configuration aspect	Example
	
Memory per executor	<pre>SPARK_PARAMETERS = { "spark.executor.memory": "8g", "spark.executor.memoryOverhead": "5g", }</pre> 
Dynamic allocation	<pre>SPARK_PARAMETERS = { "spark.dynamicAllocation.enabled": True, "spark.dynamicAllocation.initialExecutors": "2", "spark.dynamicAllocation.minExecutors": "1", "spark.dynamicAllocation.maxExecutors": "6", "spark.dynamicAllocation.executorIdleTimeout": "300s", }</pre> 

Validating Spark session configurations [🔗](#)

Once you've completed the configuration and before running a full job, verify jobs are behaving as expected. This helps spot issues early on, before committing to executing real jobs.

To do this, run the following test job:

```
spark.range(100_000_000).count()
```



Then, via the **YARN Web UI**:

- Confirm the Spark session is in the list of **Applications**.
- Monitor the Spark session **State** (running sessions should have `State = RUNNING` , while sessions that are waiting for resources should have `State = ACCEPTED`).

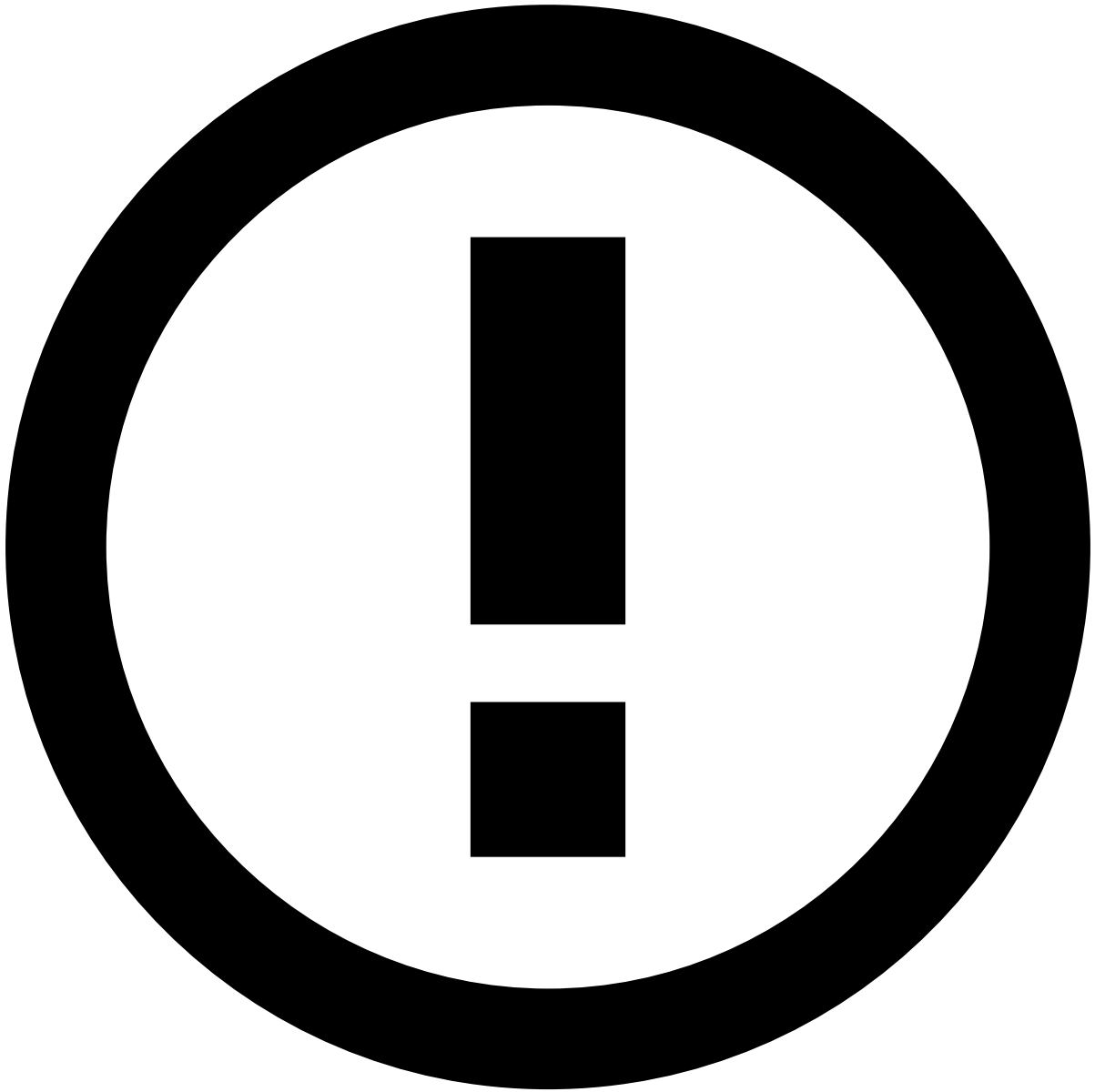
Additionally, check the **Spark History Server** to confirm that your Spark session configurations were correctly set. This can be accessed in the full list of configurations via the **Environment** tab.

Applying configurations in Pulse

For DS jobs triggered in Pulse (e.g., plan executions, model training, rule evaluations, etc.), you can use [Job Profiles](#) to provide a Spark configuration.

Read all about [Tuning Job executions with Job Profiles](#) and learn how to:

- [Create a Job Profile](#) to apply a set of property configurations when running jobs.
- [Select the profile](#) that is more suitable for the Spark job that you want to run.
- [Edit Spark properties on demand](#) to control the configuration of the Spark job that performs the calculations for a specific Data Science resource.



Jobs triggered via the DS-API

You can also trigger Pulse jobs from JupyterLab using the **DS-API**. In these cases, the job spark configurations are managed through a combination of:

- Parameters sent through the DS-API method calls.
- Pulse Job Profiles (**not** through `SPARK_PARAMETERS`).

As an example, refer to the [plan execution DS-API method documentation](#) (see the `job_profile` and `job_properties` arguments).